

KOSAC Lexicon for Social Scientists

2024-02-02 Sumin

What's done

- Load a lexicon from an existing file
 - Add `SentLex` class
- Update the lexicon from a pinch of examples
- Extract entries from a text
 - Add `Tagger` class
- Calculate the probability of each sentiment category

What's new

- Initialize and build a new lexicon from scratch
 - Add `Corpus` class
 - Rename `Tagger` class as `Tokenizer`

Demo

```
In [ ]: from utils import *
```

Loading a sentiment lexicon from a given file

```
In [ ]: kosac = SentLex(filepath='./lexicon/polarity.csv', ngrams=[1]) # Use unigram
kosac.set_lexicon(min_freq=5) # Minimum frequency 5
kosac.get_lexicon()
```

Out []:

	ngram	freq	COMP	NEG	NEUT	None	POS	max.value	max.prop
entry									
ㄴ/ETM	1	289	6	107	16	21	137	POS	0.474048
ㄴ/JX	1	22	2	8	0	2	6	NEG	0.409091
ㄴ가/EC	1	6	1	1	0	3	0	None	0.500000
ㄴ가/EF	1	9	0	2	0	5	0	None	0.555556
ㄴ다/EC	1	5	0	0	3	1	1	NEUT	0.600000
...	...	...	...	...	...	...	...	...	...
확인/NNG	1	5	0	4	0	0	1	NEG	0.800000
훨씬/MAG	1	9	0	2	0	0	6	POS	0.666667
흐르/VV	1	5	0	4	0	0	1	NEG	0.800000
흥행/NNG	1	6	0	0	0	0	6	POS	1.000000
힘/NNG	1	7	0	1	0	0	5	POS	0.857143

373 rows × 9 columns

Corpus class

In []:

```
corpus = Corpus(filepath='./data/example.csv')
corpus.df
```

Out []:

	text	label
0	나는 기뻐요.	POS
1	나는 슬퍼요.	NEG

Tokenizer class

In []:

```
sentence = '나는 슬퍼요.'
```

In []:

```
tokenizer = Tokenizer(tagger_type='Kkma') # 'Okt', 'Kkma', 'Komoran', 'Mecab'
print(tokenizer.tokenize(sentence))
print(tokenizer.get_ngrams(sentence, ns=[1,2])) # Use unigrams and bigrams
```

```
['나/NP', '는/JX', '슬프/VA', '어요/EFN', './SF']
['나/NP', '는/JX', '슬프/VA', '어요/EFN', './SF', '나/NP 는/JX', '는/JX 슬프/VA',
'슬프/VA 어요/EFN', '어요/EFN ./SF']
```

In []:

```
tokenizer = Tokenizer(tagger_type='Transformer') # Default tokenizer: KR-El
print(tokenizer.tokenize(sentence))
```

```
print(tokenizer.get_ngrams(sentence, ns=[1,2])) # Use unigrams and bigrams
```

```
['나', '##는', '슬퍼', '##요', '.']
```

```
['나', '##는', '슬퍼', '##요', '.', '나 ##는', '##는 슬퍼', '슬퍼 ##요', '##요 .']
```

SentLex class: a generalizable sentiment lexicon

Initialize an empty lexicon

```
In [ ]: sentlex = SentLex(ngrams=[1,2])
sentlex.get_lexicon()
```

Out []:

entry

Update the lexicon with a corpus and a tokenizer

```
In [ ]: sentlex.update_from_corpus(corpus, tokenizer)
sentlex.get_lexicon()
```

Out []:

	ngram	freq	COMP	NEG	NEUT	None	POS	max.value	max.prop
entry									
나	1	2	0	1	0	0	1	NEG	0.5
##는	1	2	0	1	0	0	1	NEG	0.5
기뻐	1	1	0	0	0	0	1	POS	1.0
##요	1	2	0	1	0	0	1	NEG	0.5
.	1	2	0	1	0	0	1	NEG	0.5
나 ##는	2	2	0	1	0	0	1	NEG	0.5
##는 기뻐	2	1	0	0	0	0	1	POS	1.0
기뻐 ##요	2	1	0	0	0	0	1	POS	1.0
##요 .	2	2	0	1	0	0	1	NEG	0.5
슬퍼	1	1	0	1	0	0	0	NEG	1.0
##는 슬퍼	2	1	0	1	0	0	0	NEG	1.0
슬퍼 ##요	2	1	0	1	0	0	0	NEG	1.0

TODO's

- Optimize the module
- Make a documentation

- Write a paper